

Access Modular in Java

1. public

- **Meaning:** Open for all.
- Any class, method, or variable marked as `public` can be accessed from **anywhere in the program** (same package or different package).
- **Example:**

```
public class Student {  
    public String name;  
    public void showName() {  
        System.out.println(name);  
    }  
}
```

Here, `name` and `showName()` can be used from **any other class**.

2. private

- **Meaning:** Very restricted.
- Members declared as `private` can be accessed **only inside the same class**.
- They are **not visible** outside the class, not even in child classes.
- **Example:**

```
class Student {  
    private int age;  
    private void showAge() {  
        System.out.println(age);  
    }  
}
```

`age` and `showAge()` cannot be used outside `Student`.

3. default (no keyword)

- **Meaning:** Package-level access.
- If we don't write any modifier, Java gives it **default access**.
- It can be accessed only **within the same package**, but not from other packages.

- **Example:**

```
class Student {  
    String city; // default  
    void showCity() {  
        System.out.println(city);  
    }  
}
```

`city` and `showCity()` can be used by classes in the **same package** only.

4. protected

- **Meaning:** Special case.
- Members declared as `protected` are accessible:
 - Within the **same package** (like default).
 - Also by **child classes (subclasses)**, even if they are in **different packages**.
- **Example:**

```
class Student {  
    protected String grade;  
}  
  
class Child extends Student {  
    void showGrade() {  
        System.out.println(grade); // allowed  
    }  
}
```

5. private protected

⚠️ Important Note:

- `private protected` is **not available in Java**.
- It is used in **C#**.
- In C#, it means:
 - Accessible by child classes (**inheritance**)
 - And only within the **same assembly** (like package).

Quick Comparison Table

Modifier	Same Class	Same Package	Subclass (other package)	Other Package (non-subclass)
public	✓	✓	✓	✓
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
private protected (C# only)	✓	✓ (assembly only)	✓	✗

In short:

- Use `public` → when you want everyone to access.
- Use `private` → when it's only for the same class.
- Use `default` → when only same package needs it.
- Use `protected` → when same package + subclasses should access.